雅思词汇故事网站 - 详细技术方案设计

代码块

```
// src/hooks/useAudioRecorder.ts
export function useAudioRecorder() {
  const audioUrl = ref<string>('');
  const isRecording = ref(false);
  let mediaRecorder: MediaRecorder | null = null;

  const startRecording = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    mediaRecorder = new MediaRecorder(stream);
    const chunks: Blob[] = [];

    mediaRecorder.ondataavailable = (e) => chunks.push(e.data);
    mediaRecorder.onstop = () => {
      const blob = new Blob(chunks, { type: 'audio/webm' });
      audioUrl.value = URL.createObjectURL(blob);
      stream.getTracks().forEach(t => t.stop());
    };

    mediaRecorder.start();
    isRecording.value = true;
  };

  const stopRecording = () => {
    mediaRecorder?.stop();
    isRecording.value = false;
  };

  return { audioUrl, isRecording, startRecording, stopRecording };
}
```

七、API 接口设计

统一响应格式

代码块

```
{ "code": 200, "message": "success", "data": {} }
```

接口列表

方法	路径	说明
POST	/api/auth/register	注册（含激活码校验）
POST	/api/auth/login	登录
GET	/api/auth/profile	获取用户信息
GET	/api/stories?categoryId=1	故事列表
GET	/api/stories/[id]	故事详情（含内容）
POST	/api/stories/[id]/progress	保存学习进度
GET	/api/bookmarks	收藏列表
POST	/api/bookmarks	添加收藏
DELETE	/api/bookmarks/[id]	取消收藏
POST	/api/bookmarks/batch-remove	批量取消
POST	/api/exam/start	开始考试
POST	/api/exam/submit	提交考试

注册接口示例

代码块

```
// POST /api/auth/register
// Request
{ "username": "testuser", "password": "abc12345", "activationCode": "ABCD-1234-EFGH" }

// Response 200
{ "code": 200, "data": { "userId": 1, "token": "eyJ..." } }

// Response 40001
{ "code": 40001, "message": "该激活码已被使用" }
```

八、部署流程

8.1 首次部署

代码块

```
# 1. 克隆代码
git clone https://github.com/yourname/ielts-vocab.git
cd ielts-vocab

# 2. 安装依赖
pnpm install

# 3. 创建 D1 数据库
npx wrangler d1 create ielts-vocab-db
# 将 database_id 填入 wrangler.toml

# 4. 执行数据库迁移
npx wrangler d1 migrations apply ielts-vocab-db --remote

# 5. 导入初始数据
npx wrangler d1 execute ielts-vocab-db --remote --file=./seed.sql

# 6. 部署到 Cloudflare Pages
npx wrangler pages deploy .next
# 或连接 GitHub 仓库，自动部署
```

8.2 域名配置

1. GoDaddy 购买域名（如 `ielts-vocab.com`）
2. Cloudflare 添加该域名
3. GoDaddy 修改 NS 记录为 Cloudflare 分配的 NS
4. Cloudflare DNS 添加记录：
   - `CNAME` → `ielts-vocab.pages.dev` （Cloudflare 自动分配）
5. Cloudflare 自动配置 SSL 证书

8.3 日常更新

代码块

```
git push origin main
# Cloudflare Pages 自动构建部署
```

九、缓存策略

Cloudflare 缓存层

层级	技术	用途
CDN 缓存	Cloudflare CDN	静态资源（JS/CSS/图片），自动缓存
KV 缓存	Cloudflare KV	故事列表、词汇详情等热点 API 数据
浏览器缓存	Cache-Control Header	Next.js 静态页面缓存

缓存策略

- 故事内容：KV 缓存 1 小时（内容不变，长缓存）
- 词汇数据：KV 缓存 24 小时（基本不变）
- 用户进度/收藏：不缓存，实时读写 D1
- 静态资源：Cloudflare CDN 自动处理

十、安全方案

措施	实现方式
密码加密	SHA-256 + Salt（Workers 环境，或接入 bcrypt WASM）
JWT 认证	Web Crypto API 签名，Token 放 localStorage
防暴力破解	KV 存储登录失败次数，5次失败锁定15分钟
激活码安全	同一 IP 1分钟最多尝试3次
HTTPS	Cloudflare 自动配置
XSS 防护	React 默认转义
SQL 注入	Drizzle ORM 参数化查询

十一、成本估算

项目	费用
GoDaddy 域名 (.com)	~$10/年（约 ¥72）
Cloudflare Pages	免费（500次构建/月）
Cloudflare D1	免费（5GB 存储，500万行读/天）
Cloudflare R2	免费（10GB 存储，100万次读/月）
Cloudflare KV	免费（1000次写/天，10万次读/天）
Cloudflare Workers	免费（10万次请求/天）
HTTPS 证书	免费（Cloudflare 自动）
总成本	**约 ¥72/年**

文档结束

雅思词汇故事网站 - 详细技术方案设计（最终版 V1.1）

> 配合产品需求文档使用 | 更新日期：2026-03-29

一、系统架构

代码块

```
用户（全球）
    │
    ▼ （HTTPS）
Cloudflare
    ├── DNS 解析（免费域名管理）
    ├── CDN 加速（全球节点）
    ├── Workers（运行 Next.js API）
    ├── Pages（前端静态资源）
    ├── D1（SQLite 数据库）
    └── R2（图片/音频文件存储）
```

二、技术栈

层级	技术选型	说明
域名	GoDaddy (.com)	~$10/年
DNS	Cloudflare DNS	免费
前端框架	Next.js 14 (App Router)	React 全栈框架
语言	TypeScript 5.x	类型安全
样式	Tailwind CSS 3.x	原子化 CSS，响应式优先
状态管理	Zustand	轻量
数据库	Cloudflare D1	Serverless SQLite
ORM	Drizzle ORM	TypeScript 原生，D1 支持最好
文件存储	Cloudflare R2	兼容 S3 API
缓存	Cloudflare KV	热点数据缓存
认证	JWT + Web Crypto API	Workers 环境兼容
标准发音	Web Speech API (SpeechSynthesis)	浏览器内置 TTS，系统朗读单词
语音识别	Web Speech API (SpeechRecognition)	识别用户跟读
录音回放	MediaRecorder API	录制并回放用户发音
部署	Cloudflare Pages	Git push 自动部署
HTTPS	Cloudflare 自动	免费 SSL

三、项目结构

代码块

```
ielts-vocab/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── globals.css
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   └── register/page.tsx
```

```
10  │   │   ├── (main)/
11  │   │   │   ├── layout.tsx
12  │   │   │   ├── home/page.tsx
13  │   │   │   ├── stories/
14  │   │   │   │   ├── page.tsx
15  │   │   │   │   └── [id]/page.tsx
16  │   │   │   └── bookmarks/page.tsx
17  │   │   ├── (admin)/                    # 管理后台路由组
18  │   │   │   ├── layout.tsx              # 管理后台布局
19  │   │   │   └── activation-codes/
20  │   │   │       ├── page.tsx            # 激活码列表页
21  │   │   │       └── generate/page.tsx   # 生成激活码页
22  │   │   └── api/
23  │   │       ├── auth/
24  │   │       │   ├── register/route.ts
25  │   │       │   ├── login/route.ts
26  │   │       │   └── profile/route.ts
27  │   │       ├── stories/
28  │   │       │   ├── route.ts
29  │   │       │   └── [id]/
30  │   │       │       ├── route.ts
31  │   │       │       └── progress/route.ts
32  │   │       ├── bookmarks/route.ts
33  │   │       ├── exam/route.ts
34  │   │       └── admin/                  # 管理后台 API
35  │   │           ├── activation-codes/
36  │   │           │   ├── generate/route.ts    # 批量生成激活码
37  │   │           │   └── list/route.ts        # 激活码列表查询
38  │   │           └── users/route.ts           # 用户列表
39  │   ├── components/
40  │   │   ├── WordTag.tsx
41  │   │   ├── WordPopup.tsx
42  │   │   ├── AnswerPopup.tsx
43  │   │   ├── ExamModal.tsx
44  │   │   ├── StoryCard.tsx
45  │   │   ├── ModeSwitch.tsx
46  │   │   ├── ProgressBar.tsx
47  │   │   ├── AudioWave.tsx
48  │   │   └── admin/                  # 管理后台组件
49  │   │       ├── CodeGenerator.tsx   # 激活码生成弹窗
50  │   │       └── CodeTable.tsx       # 激活码列表表格
51  │   ├── hooks/
52  │   │   ├── useAuth.ts
53  │   │   ├── useSpeech.ts            # TTS 标准发音
54  │   │   ├── useSpeechRecognition.ts
55  │   │   ├── useAudioRecorder.ts     # 录音回放
56  │   │   └── useBookmarks.ts
```

```
57  │   ├── lib/
58  │   │   ├── db/
59  │   │   │   ├── schema.ts
60  │   │   │   ├── index.ts
61  │   │   │   └── migrations/
62  │   │   ├── storyParser.ts
63  │   │   ├── wordMatcher.ts
64  │   │   ├── jwt.ts
65  │   │   ├── codeGenerator.ts        # 激活码生成工具
66  │   │   └── utils.ts
67  │   ├── stores/
68  │   │   ├── authStore.ts
69  │   │   └── storyStore.ts
70  │   └── types/
71  │       ├── vocabulary.ts
72  │       ├── story.ts
73  │       ├── exam.ts
74  │       └── activationCode.ts
75  ├── public/images/
76  ├── drizzle.config.ts
77  ├── next.config.ts
78  ├── tailwind.config.ts
79  ├── tsconfig.json
80  ├── wrangler.toml
81  └── package.json
```

四、数据库设计

4.1 Schema（Drizzle ORM）

代码块

```
1  import { sqliteTable, text, integer } from 'drizzle-orm/sqlite-core';
2
3  // 用户表
4  export const users = sqliteTable('users', {
5    id: integer('id').primaryKey({ autoIncrement: true }),
6    username: text('username').notNull().unique(),
7    passwordHash: text('password_hash').notNull(),
8    role: text('role').default('user'),           // 'user' | 'admin'
9    activationCodeId: integer('activation_code_id').references(() =>
   activationCodes.id),
10   status: integer('status').default(1),          // 1=正常 0=禁用 2=已过期
```

```
11    expiresAt: text('expires_at'),                // 账号过期时间（从激活码绑定时计
      算）
12    lastLoginAt: text('last_login_at'),
13    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
14  });
15
16  // 激活码表
17  export const activationCodes = sqliteTable('activation_codes', {
18    id: integer('id').primaryKey({ autoIncrement: true }),
19    code: text('code').notNull().unique(),        // SV-P9YF-6LVW-4YGA-Q6BZ
20    durationType: text('duration_type').notNull(), // '1min' | '24h' | '1year' |
      'forever'
21    isUsed: integer('is_used').default(0),
22    userId: integer('user_id').references(() => users.id),
23    activatedAt: text('activated_at'),             // 绑定账号的时间
24    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
25  });
26
27  // 分类表
28  export const categories = sqliteTable('categories', {
29    id: integer('id').primaryKey({ autoIncrement: true }),
30    name: text('name').notNull(),
31    slug: text('slug').notNull().unique(),
32    icon: text('icon'),
33    sortOrder: integer('sort_order').default(0),
34  });
35
36  // 故事表
37  export const stories = sqliteTable('stories', {
38    id: integer('id').primaryKey({ autoIncrement: true }),
39    categoryId: integer('category_id').notNull().references(() => categories.id),
40    title: text('title').notNull(),
41    description: text('description'),
42    content: text('content').notNull(),
43    coverUrl: text('cover_url'),
44    sourceUrl: text('source_url'),
45    sourceName: text('source_name'),
46    wordCount: integer('word_count').default(0),
47    sortOrder: integer('sort_order').default(0),
48    status: integer('status').default(1),
49    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
50  });
51
52  // 词汇表
53  export const vocabularies = sqliteTable('vocabularies', {
54    id: integer('id').primaryKey({ autoIncrement: true }),
55    word: text('word').notNull().unique(),
```

```
56    phonetic: text('phonetic'),
57    translation: text('translation').notNull(),
58    difficulty: integer('difficulty').default(1),
59    frequency: integer('frequency').default(0),
60  });
61
62  // 故事-词汇关联表
63  export const storyWords = sqliteTable('story_words', {
64    id: integer('id').primaryKey({ autoIncrement: true }),
65    storyId: integer('story_id').notNull().references(() => stories.id),
66    wordId: integer('word_id').notNull().references(() => vocabularies.id),
67    position: integer('position').default(0),
68    sentenceText: text('sentence_text'),
69  });
70
71  // 收藏表
72  export const bookmarks = sqliteTable('bookmarks', {
73    id: integer('id').primaryKey({ autoIncrement: true }),
74    userId: integer('user_id').notNull().references(() => users.id),
75    wordId: integer('word_id').notNull().references(() => vocabularies.id),
76    storyId: integer('story_id').references(() => stories.id),
77    masteryLevel: integer('mastery_level').default(0),
78    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
79  });
80
81  // 学习进度表
82  export const learningProgress = sqliteTable('learning_progress', {
83    id: integer('id').primaryKey({ autoIncrement: true }),
84    userId: integer('user_id').notNull().references(() => users.id),
85    storyId: integer('story_id').notNull().references(() => stories.id),
86    mode: text('mode').notNull(),
87    completedWords: text('completed_words').default('[]'),
88    correctCount: integer('correct_count').default(0),
89    wrongCount: integer('wrong_count').default(0),
90    scrollPosition: integer('scroll_position').default(0),
91    lastReadAt: text('last_read_at').default(sql`CURRENT_TIMESTAMP`),
92  });
93
94  // 考试记录表
95  export const examRecords = sqliteTable('exam_records', {
96    id: integer('id').primaryKey({ autoIncrement: true }),
97    userId: integer('user_id').notNull().references(() => users.id),
98    storyId: integer('story_id').notNull().references(() => stories.id),
99    totalQuestions: integer('total_questions').default(10),
100   correctCount: integer('correct_count').default(0),
101   wrongWords: text('wrong_words').default('[]'),
102   accuracy: integer('accuracy').default(0),
```

```
  durationSeconds: integer('duration_seconds').default(0),
  createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
});
```

4.2 激活码时效计算逻辑

代码块

```
激活码绑定账号时，根据 durationType 计算账号过期时间：

durationType = '1min'   → expiresAt = activatedAt + 1分钟
durationType = '24h'    → expiresAt = activatedAt + 24小时
durationType = '1year'  → expiresAt = activatedAt + 365天
durationType = 'forever' → expiresAt = null（永不过期）
```

4.3 登录过期校验逻辑

代码块

```
用户登录时：
1. 校验用户名密码
2. 检查 users.status：
   - status = 0 → "账号已被禁用"
   - status = 2 → "账号已过期"
3. 检查 expiresAt：
   - 如果 expiresAt 不为空 且 当前时间 > expiresAt
     → 更新 status = 2，返回"账号已过期"
4. 通过后签发 JWT Token
```

五、激活码生成工具

5.1 激活码格式

格式：`SV-XXXX-XXXX-XXXX-XXXX`

- 前缀：`SV-`（固定）
- 4段：每段4位大写字母+数字随机组合
- 总长度：20 字符

5.2 生成算法

```typo_code_generator_ts
// src/lib/codeGenerator.ts

const CHARS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';

function randomSegment(): string {
  let result = '';
  for (let i = 0; i < 4; i++) {
    result += CHARS[Math.floor(Math.random() * CHARS.length)];
  }
  return result;
}

export function generateActivationCode(): string {
  return `SV-${randomSegment()}-${randomSegment()}-${randomSegment()}-${randomSegment()}`;
}

export async function batchGenerateCodes(
  db: D1Database,
  count: number,
  durationType: '1min' | '24h' | '1year' | 'forever'
): Promise<string[]> {
  const codes: string[] = [];
  const existing = new Set<string>();

  for (let i = 0; i < count; i++) {
    let code: string;
    do {
      code = generateActivationCode();
    } while (existing.has(code)); // 确保本次批量内唯一

    existing.add(code);
    codes.push(code);
  }

  // 批量写入数据库
  await db.batch(
    codes.map(code => ({
      sql: 'INSERT INTO activation_codes (code, duration_type) VALUES (?, ?)',
      args: [code, durationType],
    }))
  );

  return codes;
}
```

六、核心代码实现

6.1 语音合成 — 标准发音（TTS）

代码块

```
// src/hooks/useSpeech.ts
// 使用浏览器内置 TTS，系统自动朗读单词的标准英文发音
// 效果类似有道词典的"小喇叭"发音按钮

export function useSpeech() {
  const isSpeaking = ref(false);

  const speak = (word: string) => {
    if (!('speechSynthesis' in window)) {
      message.warning('您的浏览器不支持语音合成');
      return;
    }

    speechSynthesis.cancel(); // 停止上一个发音

    const utterance = new SpeechSynthesisUtterance(word);
    utterance.lang = 'en-US';
    utterance.rate = 0.8;    // 稍慢，便于学习者听清
    utterance.pitch = 1;

    // 优先选择英语母语语音
    const voices = speechSynthesis.getVoices();
    const enVoice = voices.find(v =>
      v.lang === 'en-US' || v.lang.startsWith('en')
    );
    if (enVoice) utterance.voice = enVoice;

    utterance.onstart = () => isSpeaking.value = true;
    utterance.onend = () => isSpeaking.value = false;

    speechSynthesis.speak(utterance);
  };

  const stop = () => {
    speechSynthesis.cancel();
    isSpeaking.value = false;
  };

  return { speak, stop, isSpeaking };
```

```
}
```

用户体验流程：

1. 用户在故事中点击词汇 → 弹出单词详情弹窗
2. 弹窗中点击 🔊 发音按钮
3. 浏览器调用 TTS 引擎，自动朗读该单词的标准英文发音
4. 用户听到发音（就像有道词典一样）
5. 按钮显示音波动画表示正在播放

6.2 语音识别 + 录音回放（跟读对比）

代码块

```
// src/hooks/useSpeechRecognition.ts
// 用户点击🎤读词按钮 → 朗读单词 → 系统识别 + 录音

export function useSpeechRecognition() {
  const isRecording = ref(false);
  const recognizedText = ref('');

  const startRecognition = (onResult?: (text: string) => void) => {
    if (!('webkitSpeechRecognition' in window) &&
        !('SpeechRecognition' in window)) {
      message.warning('您的浏览器不支持语音识别');
      return;
    }

    const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
    const recognition = new SpeechRecognition();
    recognition.lang = 'en-US';
    recognition.continuous = false;
    recognition.interimResults = false;

    recognition.onresult = (event) => {
      recognizedText.value = event.results[0][0].transcript;
      onResult?.(recognizedText.value);
    };

    recognition.onend = () => isRecording.value = false;
    recognition.start();
    isRecording.value = true;
  };

```

```
  return { isRecording, recognizedText, startRecognition };
}

// src/hooks/useAudioRecorder.ts
// 录制用户发音，生成可回放的音频按钮

export function useAudioRecorder() {
  const audioUrl = ref<string>('');
  const isRecording = ref(false);
  let mediaRecorder: MediaRecorder | null = null;

  const startRecording = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    mediaRecorder = new MediaRecorder(stream);
    const chunks: Blob[] = [];

    mediaRecorder.ondataavailable = (e) => chunks.push(e.data);
    mediaRecorder.onstop = () => {
      const blob = new Blob(chunks, { type: 'audio/webm' });
      audioUrl.value = URL.createObjectURL(blob);
      stream.getTracks().forEach(t => t.stop());
    };

    mediaRecorder.start();
    isRecording.value = true;
  };

  const stopRecording = () => {
    mediaRecorder?.stop();
    isRecording.value = false;
  };

  return { audioUrl, isRecording, startRecording, stopRecording };
}
```

6.3 故事文本解析器

代码块

```
// src/lib/storyParser.ts

export interface StorySegment {
  type: 'text' | 'word';
  content: string;
  translation?: string;
  position?: number;
```

```
}

export function parseStoryContent(rawText: string): StorySegment[] {
  const regex = /([a-zA-Z]+)\s*\[([^\]]+)\]/g;
  const segments: StorySegment[] = [];
  let lastIndex = 0;
  let match;
  let position = 0;

  while ((match = regex.exec(rawText)) !== null) {
    if (match.index > lastIndex) {
      segments.push({ type: 'text', content: rawText.slice(lastIndex, match.index) });
    }
    segments.push({
      type: 'word',
      content: match[1],
      translation: match[2],
      position: position++,
    });
    lastIndex = regex.lastIndex;
  }

  if (lastIndex < rawText.length) {
    segments.push({ type: 'text', content: rawText.slice(lastIndex) });
  }

  return segments;
}
```

6.4 答案匹配逻辑

代码块

```
// src/lib/wordMatcher.ts

export function matchChinese(input: string, answer: string): boolean {
  const normalize = (s: string) => s.trim().replace(/[, 。、 ! ? ; : ""'' () \s]/g, '');
  const i = normalize(input);
  const a = normalize(answer);
  if (i === a) return true;
  if (i.length >= 2 && (a.includes(i) || i.includes(a))) return true;
  return false;
}

```

```
export function matchEnglish(input: string, answer: string): boolean {
  return input.trim().toLowerCase() === answer.trim().toLowerCase();
}
```

6.5 JWT 工具（Web Crypto API）

代码块

```
// src/lib/jwt.ts

async function hashPassword(password: string): Promise<string> {
  const encoder = new TextEncoder();
  const hashBuffer = await crypto.subtle.digest(
    'SHA-256',
    encoder.encode(password + 'salt')
  );
  return Array.from(new Uint8Array(hashBuffer))
    .map(b => b.toString(16).padStart(2, '0')).join('');
}

async function signJWT(payload: object, secret: string): Promise<string> {
  const encoder = new TextEncoder();
  const header = btoa(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));
  const body = btoa(JSON.stringify(payload));

  const key = await crypto.subtle.importKey(
    'raw', encoder.encode(secret),
    { name: 'HMAC', hash: 'SHA-256' },
    false, ['sign']
  );

  const signature = await crypto.subtle.sign(
    'HMAC', key, encoder.encode(`${header}.${body}`)
  );

  const sig = btoa(String.fromCharCode(...new Uint8Array(signature)));
  return `${header}.${body}.${sig}`;
}
```

七、API 接口设计

统一响应格式

```
代码块
{ "code": 200, "message": "success", "data": {} }
```

7.1 用户认证接口

方法	路径	说明
POST	/api/auth/register	注册（含激活码校验 + 时效计算）
POST	/api/auth/login	登录（含过期校验）
GET	/api/auth/profile	获取用户信息

注册接口 — 激活码校验 + 时效计算：

```
代码块
// POST /api/auth/register
// Request
{ "username": "testuser", "password": "abc12345", "activationCode": "SV-P9YF-6LVW-4YGA-Q6BZ" }

// 后端逻辑:
// 1. 查询 activation_codes 表，检查 code 是否存在
// 2. 检查 is_used 是否为 0
// 3. 根据 durationType 计算 expiresAt
// 4. 创建用户，绑定激活码，标记 is_used = 1

// Response 200
{ "code": 200, "data": { "userId": 1, "token": "eyJ..." } }

// Response 40001 — 激活码已使用
{ "code": 40001, "message": "该激活码已被使用" }

// Response 40002 — 激活码不存在
{ "code": 40002, "message": "激活码无效" }
```

登录接口 — 过期校验：

```
代码块
// POST /api/auth/login
// 后端逻辑:
// 1. 校验用户名密码
// 2. 检查 status === 0 → "账号已被禁用"
```

```
// 3. 检查 status === 2 → "账号已过期"
// 4. 检查 expiresAt && now > expiresAt → 更新 status=2, "账号已过期"
// 5. 签发 JWT

// Response 20003 — 账号过期
{ "code": 20003, "message": "您的账号已过期，请联系客服" }
```

7.2 故事 + 收藏 + 考试接口

方法	路径	说明
GET	/api/stories?categoryId=1	故事列表
GET	/api/stories/[id]	故事详情
POST	/api/stories/[id]/progress	保存进度
GET	/api/bookmarks	收藏列表
POST	/api/bookmarks	添加收藏
DELETE	/api/bookmarks/[id]	取消收藏
POST	/api/bookmarks/batch-remove	批量取消
POST	/api/exam/start	开始考试
POST	/api/exam/submit	提交考试

7.3 管理后台接口（需管理员权限）

方法	路径	说明
POST	/api/admin/activation-codes/generate	批量生成激活码
GET	/api/admin/activation-codes/list	激活码列表（分页+筛选）
GET	/api/admin/users	用户列表

生成激活码接口：

代码块

```
// POST /api/admin/activation-codes/generate
// Request
```

```
{
  "count": 1000,
  "durationType": "24h"     // "1min" | "24h" | "1year" | "forever"
}

// Response 200
{
  "code": 200,
  "data": {
    "generatedCount": 1000,
    "durationType": "24h",
    "codes": [
      "SV-P9YF-6LVW-4YGA-Q6BZ",
      "SV-A3KJ-9MNX-2PLC-7HRW",
      "..."
    ]
  }
}
```

激活码列表查询接口：

代码块

```
// GET /api/admin/activation-codes/list
// Query 参数:
//   page: number (默认 1)
//   pageSize: number (默认 20)
//   username: string (可选, 按账号模糊搜索)
//   code: string (可选, 按激活码搜索)
//   startDate: string (可选, 激活开始时间 YYYY-MM-DD)
//   endDate: string (可选, 激活结束时间 YYYY-MM-DD)
//   durationType: string (可选, 1min/24h/1year/forever)
//   status: string (可选, unused/activated/expired)

// Response 200
{
  "code": 200,
  "data": {
    "total": 5000,
    "page": 1,
    "pageSize": 20,
    "items": [
      {
        "id": 1,
        "code": "SV-P9YF-6LVW-4YGA-Q6BZ",
        "username": "testuser01",
```

```
          "password": "ab***45",           // 脱敏显示
          "activatedAt": "2026-03-29 10:30:00",
          "expiresAt": "2026-03-30 10:30:00",
          "durationType": "24h",
          "durationLabel": "24小时",
          "status": "activated"
        }
      ]
    }
  }
```

八、管理后台页面设计

8.1 激活码生成弹窗

代码块

```
┌──────────────────────────────────────────┐
│        生成激活码                         │
│                                          │
│  生成数量:                                │
│  ┌──────────────────────┐                │
│  │ 1000                 │                │
│  └──────────────────────┘                │
│                                          │
│  时效类型:                                │
│  ○ 一分钟                                 │
│  ● 24小时                                 │
│  ○ 一年                                   │
│  ○ 永久                                   │
│                                          │
│        [ 取消 ]    [ 生成 ]               │
└──────────────────────────────────────────┘
```

生成后展示结果弹窗，每个激活码一行，提供"复制全部"和"导出TXT"按钮。

8.2 激活码列表页

顶部：搜索筛选栏

筛选项	类型	说明

账号搜索	输入框	模糊搜索用户名
激活码搜索	输入框	精确/模糊搜索
激活时间段	日期选择器	开始 - 结束
账号类型	下拉	一分钟/24小时/一年/永久
状态	下拉	未使用/已激活/已过期

列表表格：

激活码	账号	密码	激活时间	失效时间	账号类型	状态
SV-P9YF-...	testuser01	ab***45	2026-03-29 10:30	2026-03-30 10:30	24小时	已激活
SV-A3KJ-...	-	-	-	-	一年	未使用

底部：分页器（上一页 1 2 3 ... 50 下一页）

九、Cloudflare 配置

代码块

```
# wrangler.toml
name = "ielts-vocab"
compatibility_date = "2024-01-01"

[[d1_databases]]
binding = "DB"
database_name = "ielts-vocab-db"
database_id = "xxxx-xxxx-xxxx"

[[kv_namespaces]]
binding = "CACHE"
id = "xxxx-xxxx-xxxx"

[[r2_buckets]]
binding = "STORAGE"
bucket_name = "ielts-vocab-assets"
```

十、部署流程

代码块

```
# 首次部署
git clone https://github.com/yourname/ielts-vocab.git
cd ielts-vocab && pnpm install

# 创建 D1 数据库
npx wrangler d1 create ielts-vocab-db

# 执行数据库迁移
npx wrangler d1 migrations apply ielts-vocab-db --remote

# 导入初始数据（词汇 + 管理员账号 + 分类）
npx wrangler d1 execute ielts-vocab-db --remote --file=./seed.sql

# 部署到 Cloudflare Pages
npx wrangler pages deploy .next

# 域名配置
# GoDaddy 购买域名 → Cloudflare 添加域名 → 修改 NS → CNAME 指向 pages.dev
```

十一、安全方案

措施	实现方式
密码加密	SHA-256 + Salt
JWT 认证	Web Crypto API 签名
管理员权限	JWT payload 中包含 role，中间件校验 role='admin'
防暴力破解	KV 存储登录失败次数，5次锁定15分钟
激活码安全	同一 IP 1分钟最多尝试3次
账号过期	登录时校验 expiresAt，过期则拒绝并更新状态
HTTPS	Cloudflare 自动
SQL 注入	Drizzle ORM 参数化查询

十二、成本估算

项目	费用
GoDaddy 域名	~$10/年（约 ¥72）
Cloudflare 全家桶	免费
HTTPS 证书	免费
总成本	**约 ¥72/年**

文档结束

雅思词汇故事网站 - 详细技术方案设计（最终版 V1.2）

> 配合产品需求文档使用 | 更新日期：2026-03-29

一、系统架构

代码块

```
用户（全球）
    │
    ▼ （HTTPS）
Cloudflare
    ├── DNS 解析（GoDaddy 域名）
    ├── CDN 加速（全球节点）
    ├── Workers（运行 Next.js API）
    ├── Pages（前端静态资源）
    ├── D1（SQLite 数据库）
    └── R2（音频 MP3 文件 + 图片存储）
```

二、技术栈

层级	技术选型	说明

域名	GoDaddy (.com)	~$10/年
DNS	Cloudflare DNS	免费
前端框架	Next.js 14 (App Router)	React 全栈框架
语言	TypeScript 5.x	类型安全
样式	Tailwind CSS 3.x	原子化 CSS，响应式优先
状态管理	Zustand	轻量
数据库	Cloudflare D1	Serverless SQLite
ORM	Drizzle ORM	TypeScript 原生，D1 支持最好
文件存储	Cloudflare R2	音频 MP3 + 图片，兼容 S3 API
缓存	Cloudflare KV	热点数据缓存
认证	JWT + Web Crypto API	Workers 环境兼容
发音音频	edge-tts 预生成 MP3	美式 AriaNeural + 英式 SoniaNeural
音标数据	Free Dictionary API	批量生成，存数据库
音频加载策略	懒加载	打开故事时按需加载该故事 100 个单词音频
跟读录音	MediaRecorder API	浏览器原生，纯本地录制
语音识别	Web Speech API (SpeechRecognition)	识别用户跟读
XLS 导出	xlsx (SheetJS)	管理后台导出激活码
部署	Cloudflare Pages	Git push 自动部署
HTTPS	Cloudflare 自动	免费 SSL

三、项目结构

代码块

```
ielts-vocab/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx
```

```
6   │   │   ├── globals.css
7   │   │   ├── (auth)/
8   │   │   │   ├── login/page.tsx
9   │   │   │   └── register/page.tsx
10  │   │   ├── (main)/
11  │   │   │   ├── layout.tsx
12  │   │   │   ├── home/page.tsx
13  │   │   │   ├── stories/
14  │   │   │   │   ├── page.tsx
15  │   │   │   │   └── [id]/page.tsx
16  │   │   │   └── bookmarks/page.tsx
17  │   │   ├── (admin)/
18  │   │   │   ├── layout.tsx
19  │   │   │   └── activation-codes/
20  │   │   │       ├── page.tsx
21  │   │   │       └── generate/page.tsx
22  │   │   └── api/
23  │   │       ├── auth/
24  │   │       │   ├── register/route.ts
25  │   │       │   ├── login/route.ts
26  │   │       │   └── profile/route.ts
27  │   │       ├── stories/
28  │   │       │   ├── route.ts
29  │   │       │   └── [id]/
30  │   │       │       ├── route.ts
31  │   │       │       └── progress/route.ts
32  │   │       ├── bookmarks/route.ts
33  │   │       ├── exam/route.ts
34  │   │       └── admin/
35  │   │           ├── activation-codes/
36  │   │           │   ├── generate/route.ts
37  │   │           │   └── list/route.ts
38  │   │           └── users/route.ts
39  │   ├── components/
40  │   │   ├── WordTag.tsx
41  │   │   ├── WordPopup.tsx
42  │   │   ├── AnswerPopup.tsx
43  │   │   ├── ExamModal.tsx
44  │   │   ├── StoryCard.tsx
45  │   │   ├── ModeSwitch.tsx
46  │   │   ├── ProgressBar.tsx
47  │   │   ├── AudioWave.tsx
48  │   │   ├── AccentSwitch.tsx          # 美式/英式切换组件
49  │   │   └── admin/
50  │   │       ├── CodeGenerator.tsx
51  │   │       └── CodeTable.tsx
52  │   ├── hooks/
```

```
53  │   │   ├── useAuth.ts
54  │   │   ├── usePronunciation.ts        # 播放 R2 音频（替代 TTS）
55  │   │   ├── useSpeechRecognition.ts
56  │   │   ├── useAudioRecorder.ts
57  │   │   └── useBookmarks.ts
58  │   ├── lib/
59  │   │   ├── db/
60  │   │   │   ├── schema.ts
61  │   │   │   ├── index.ts
62  │   │   │   └── migrations/
63  │   │   ├── storyParser.ts
64  │   │   ├── wordMatcher.ts
65  │   │   ├── jwt.ts
66  │   │   ├── codeGenerator.ts
67  │   │   └── utils.ts
68  │   ├── stores/
69  │   │   ├── authStore.ts
70  │   │   └── storyStore.ts
71  │   └── types/
72  ├── scripts/                          # 数据准备脚本（开发阶段用）
73  │   ├── generate-phonetics.py         # 批量生成音标（Free Dictionary API）
74  │   ├── generate-audio.py             # 批量生成 MP3（edge-tts）
75  │   └── upload-audio.sh               # 上传音频到 R2
76  ├── public/images/
77  ├── drizzle.config.ts
78  ├── next.config.ts
79  ├── tailwind.config.ts
80  ├── tsconfig.json
81  ├── wrangler.toml
82  └── package.json
```

四、数据库设计

4.1 Schema（Drizzle ORM）

代码块

```
1  import { sqliteTable, text, integer } from 'drizzle-orm/sqlite-core';
2
3  // 用户表
4  export const users = sqliteTable('users', {
5    id: integer('id').primaryKey({ autoIncrement: true }),
6    username: text('username').notNull().unique(),
7    passwordHash: text('password_hash').notNull(),
```

```
8     role: text('role').default('user'),           // 'user' | 'admin'
9     activationCodeId: integer('activation_code_id').references(() =>
    activationCodes.id),
10    status: integer('status').default(1),          // 1=正常 0=禁用 2=已过期
11    expiresAt: text('expires_at'),                // 账号过期时间
12    lastLoginAt: text('last_login_at'),
13    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
14  });
15
16  // 激活码表
17  export const activationCodes = sqliteTable('activation_codes', {
18    id: integer('id').primaryKey({ autoIncrement: true }),
19    code: text('code').notNull().unique(),        // SV-P9YF-6LVW-4YGA-Q6BZ
20    durationType: text('duration_type').notNull(), // '1min' | '24h' | '1year' |
    'forever'
21    isUsed: integer('is_used').default(0),
22    userId: integer('user_id').references(() => users.id),
23    activatedAt: text('activated_at'),
24    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
25  });
26
27  // 分类表
28  export const categories = sqliteTable('categories', {
29    id: integer('id').primaryKey({ autoIncrement: true }),
30    name: text('name').notNull(),
31    slug: text('slug').notNull().unique(),
32    icon: text('icon'),
33    sortOrder: integer('sort_order').default(0),
34  });
35
36  // 故事表
37  export const stories = sqliteTable('stories', {
38    id: integer('id').primaryKey({ autoIncrement: true }),
39    categoryId: integer('category_id').notNull().references(() => categories.id),
40    title: text('title').notNull(),
41    description: text('description'),
42    content: text('content').notNull(),
43    coverUrl: text('cover_url'),
44    sourceUrl: text('source_url'),
45    sourceName: text('source_name'),
46    wordCount: integer('word_count').default(0),
47    sortOrder: integer('sort_order').default(0),
48    status: integer('status').default(1),
49    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
50  });
51
52  // 词汇表
```

```
53  export const vocabularies = sqliteTable('vocabularies', {
54    id: integer('id').primaryKey({ autoIncrement: true }),
55    word: text('word').notNull().unique(),
56    phoneticUs: text('phonetic_us'),              // 美式音标 /hɑːt/
57    phoneticUk: text('phonetic_uk'),              // 英式音标 /hɒt/
58    translation: text('translation').notNull(),
59    difficulty: integer('difficulty').default(1),
60    frequency: integer('frequency').default(0),
61  });
62
63  // 故事-词汇关联表
64  export const storyWords = sqliteTable('story_words', {
65    id: integer('id').primaryKey({ autoIncrement: true }),
66    storyId: integer('story_id').notNull().references(() => stories.id),
67    wordId: integer('word_id').notNull().references(() => vocabularies.id),
68    position: integer('position').default(0),
69    sentenceText: text('sentence_text'),
70  });
71
72  // 收藏表
73  export const bookmarks = sqliteTable('bookmarks', {
74    id: integer('id').primaryKey({ autoIncrement: true }),
75    userId: integer('user_id').notNull().references(() => users.id),
76    wordId: integer('word_id').notNull().references(() => vocabularies.id),
77    storyId: integer('story_id').references(() => stories.id),
78    masteryLevel: integer('mastery_level').default(0),
79    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
80  });
81
82  // 学习进度表
83  export const learningProgress = sqliteTable('learning_progress', {
84    id: integer('id').primaryKey({ autoIncrement: true }),
85    userId: integer('user_id').notNull().references(() => users.id),
86    storyId: integer('story_id').notNull().references(() => stories.id),
87    mode: text('mode').notNull(),
88    completedWords: text('completed_words').default('[]'),
89    correctCount: integer('correct_count').default(0),
90    wrongCount: integer('wrong_count').default(0),
91    scrollPosition: integer('scroll_position').default(0),
92    lastReadAt: text('last_read_at').default(sql`CURRENT_TIMESTAMP`),
93  });
94
95  // 考试记录表
96  export const examRecords = sqliteTable('exam_records', {
97    id: integer('id').primaryKey({ autoIncrement: true }),
98    userId: integer('user_id').notNull().references(() => users.id),
99    storyId: integer('story_id').notNull().references(() => stories.id),
```

```
    totalQuestions: integer('total_questions').default(10),
    correctCount: integer('correct_count').default(0),
    wrongWords: text('wrong_words').default('[]'),
    accuracy: integer('accuracy').default(0),
    durationSeconds: integer('duration_seconds').default(0),
    createdAt: text('created_at').default(sql`CURRENT_TIMESTAMP`),
  });
```

4.2 激活码时效与登录过期校验

绑定账号时计算过期时间：

代码块

```
durationType = '1min'   → expiresAt = activatedAt + 1分钟
durationType = '24h'    → expiresAt = activatedAt + 24小时
durationType = '1year'  → expiresAt = activatedAt + 365天
durationType = 'forever' → expiresAt = null（永不过期）
```

登录时校验：

代码块

```
1. 校验用户名密码
2. status = 0 → "账号已被禁用"
3. status = 2 → "账号已过期"
4. expiresAt 不为空 && now > expiresAt → 更新 status=2, "账号已过期"
5. 通过后签发 JWT
```

五、发音与音标方案

5.1 发音音频方案（R2 静态文件 + 懒加载）

为什么不用浏览器 TTS 或第三方 API：

- 浏览器 SpeechSynthesis 音质不稳定，不同浏览器差异大
- Google TTS 国内无法访问
- 有道智云每日 200 次免费额度太少
- 第三方 API 有调用限制和网络依赖

方案：edge-tts 预生成 + R2 存储

代码块

```
生成阶段（一次性，开发环境）：
  edge-tts → 9400 个单词 × 2 种发音 = 18800 个 MP3
  → 上传到 Cloudflare R2

运行阶段（用户访问时）：
  用户打开故事 → 前端拿到该故事 100 个单词列表
  → 后台静默预加载 100 个 MP3（不影响页面渲染）
  → 用户点击🔊发音 → 直接播放本地缓存的音频
  → 体验：即时播放，无延迟
```

R2 文件结构：

代码块

```
ielts-vocab-assets/
└── audio/
    ├── us/                    # 美式发音
    │   ├── accuse.mp3
    │   ├── hot.mp3
    │   └── ...
    └── uk/                    # 英式发音
        ├── accuse.mp3
        ├── hot.mp3
        └── ...
```

懒加载策略：

代码块

```
// src/hooks/usePronunciation.ts

// 音频基础 URL（R2 公开访问域名）
const AUDIO_BASE_URL = 'https://your-r2-domain/audio';

// 当前故事已预加载的单词音频缓存
const audioCache = new Map<string, HTMLAudioElement>();

// 用户打开故事时，预加载该故事 100 个单词的音频
export function preloadStoryAudio(words: string[], accent: 'us' | 'uk') {
  words.forEach(word => {
    const url = `${AUDIO_BASE_URL}/${accent}/${word}.mp3`;
    const audio = new Audio(url);
    audio.preload = 'auto';  // 后台静默下载
    audioCache.set(`${accent}:${word}`, audio);
  });
}
```

```

// 用户点击发音按钮时播放
export function playPronunciation(word: string, accent: 'us' | 'uk') {
  const audio = audioCache.get(`${accent}:${word}`);
  if (audio) {
    audio.currentTime = 0;  // 从头播放
    audio.play();
  }
}
```

存储空间估算：

- 每个单词 MP3：约 5-10KB
- 9400 × 2（美英） × 7KB（平均）≈ **130MB**
- Cloudflare R2 免费额度 10GB，仅用 1.3%

5.2 音标生成方案

工具：Free Dictionary API（免费、无需注册）

代码块

```
API 地址：https://api.dictionaryapi.dev/api/v2/entries/en/{word}

返回示例（查 "hot"）：
{
  "phonetics": [
    { "text": "/hɑːt/", "audio": "..." },   // 美式
    { "text": "/hɒt/", "audio": "..." }      // 英式
  ]
}
```

批量生成脚本：

代码块

```
# scripts/generate-phonetics.py

import requests
import csv
import time

API_URL = "https://api.dictionaryapi.dev/api/v2/entries/en/"

def get_phonetics(word):
```

```
    try:
        resp = requests.get(f"{API_URL}{word}", timeout=5)
        data = resp.json()
        phonetic_us = ""
        phonetic_uk = ""
        for entry in data:
            for p in entry.get("phonetics", []):
                text = p.get("text", "")
                audio = p.get("audio", "")
                if audio and "us" in audio:
                    phonetic_us = text
                elif audio and "uk" in audio:
                    phonetic_uk = text
                elif text and not phonetic_us:
                    phonetic_us = text
        return phonetic_us, phonetic_uk
    except:
        return "", ""

# 从词汇 CSV 读取，逐个查询，写入新 CSV
# 约 9400 个单词，每个间隔 0.2 秒，约 30 分钟完成
```

5.3 音频生成脚本

代码块

```
# scripts/generate-audio.py

import asyncio
import edge_tts
import os

# 美式发音
US_VOICE = "en-US-AriaNeural"
# 英式发音
UK_VOICE = "en-GB-SoniaNeural"

async def generate_word_audio(word, voice, output_dir):
    output_path = os.path.join(output_dir, f"{word}.mp3")
    if os.path.exists(output_path):
        return  # 跳过已生成
    communicate = edge_tts.Communicate(word, voice)
    await communicate.save(output_path)

async def batch_generate(words, voice, output_dir):
    os.makedirs(output_dir, exist_ok=True)
```

```
    for word in words:
        await generate_word_audio(word, voice, output_dir)

# 从词汇 CSV 读取单词列表，分别生成美式和英式
# 9400 个单词 × 2 ≈ 2 小时
```

六、激活码生成方案

格式： `SV-XXXX-XXXX-XXXX-XXXX`

代码块

```
// src/lib/codeGenerator.ts

const CHARS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';

function randomSegment(): string {
  let result = '';
  for (let i = 0; i < 4; i++) {
    result += CHARS[Math.floor(Math.random() * CHARS.length)];
  }
  return result;
}

export function generateActivationCode(): string {
  return `SV-${randomSegment()}-${randomSegment()}-${randomSegment()}-${randomSegment()}`;
}

export async function batchGenerateCodes(
  db: D1Database,
  count: number,
  durationType: '1min' | '24h' | '1year' | 'forever'
): Promise<string[]> {
  const codes: string[] = [];
  const existing = new Set<string>();

  for (let i = 0; i < count; i++) {
    let code: string;
    do { code = generateActivationCode(); } while (existing.has(code));
    existing.add(code);
    codes.push(code);
  }

```

```
  await db.batch(
    codes.map(code => ({
      sql: 'INSERT INTO activation_codes (code, duration_type) VALUES (?, ?)',
      args: [code, durationType],
    }))
  );

  return codes;
}
```

XLS 导出：

代码块

```
// 管理后台 API - 导出激活码为 XLS
import * as XLSX from 'xlsx';

export async function exportActivationCodes(codes: string[], durationType: string) {
  const wb = XLSX.utils.book_new();
  const data = codes.map(code => ({ '激活码': code }));
  const ws = XLSX.utils.json_to_sheet(data);
  XLSX.utils.book_append_sheet(wb, ws, '激活码');

  const buffer = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
  const timestamp = new Date().toISOString().split('T')[0];
  const typeLabel = { '1min': '一分钟', '24h': '24小时', '1year': '一年', 'forever': '永久' }[durationType];

  return {
    buffer,
    filename: `激活码_${timestamp}_${typeLabel}_${codes.length}个.xlsx`
  };
}
```

七、核心前端代码

7.1 发音 Hook（R2 音频播放）

代码块

```
// src/hooks/usePronunciation.ts

```

```
3   const AUDIO_BASE_URL = 'https://your-r2-domain/audio';
4   const audioCache = new Map<string, HTMLAudioElement>();
5
6   export function usePronunciation() {
7     const isPlaying = ref(false);
8     const accent = ref<'us' | 'uk'>('us');  // 默认美式
9
10    // 预加载故事音频
11    const preload = (words: string[]) => {
12      words.forEach(word => {
13        const url = `${AUDIO_BASE_URL}/${accent.value}/${word}.mp3`;
14        const audio = new Audio(url);
15        audio.preload = 'auto';
16        audioCache.set(`${accent.value}:${word}`, audio);
17      });
18    };
19
20    // 播放发音
21    const play = (word: string) => {
22      const audio = audioCache.get(`${accent.value}:${word}`);
23      if (!audio) {
24        // 未缓存时直接创建
25        const url = `${AUDIO_BASE_URL}/${accent.value}/${word}.mp3`;
26        const audio = new Audio(url);
27        audio.play();
28        return;
29      }
30      audio.currentTime = 0;
31      audio.onplay = () => isPlaying.value = true;
32      audio.onended = () => isPlaying.value = false;
33      audio.play();
34    };
35
36    // 切换美式/英式
37    const toggleAccent = () => {
38      accent.value = accent.value === 'us' ? 'uk' : 'us';
39    };
40
41    return { play, preload, accent, isPlaying, toggleAccent };
42  }
```

7.2 故事文本解析器

代码块

```
1   // src/lib/storyParser.ts
```

```

export interface StorySegment {
  type: 'text' | 'word';
  content: string;
  translation?: string;
  position?: number;
}

export function parseStoryContent(rawText: string): StorySegment[] {
  const regex = /([a-zA-Z]+)\s*\[([^\]]+)\]/g;
  const segments: StorySegment[] = [];
  let lastIndex = 0;
  let match;
  let position = 0;

  while ((match = regex.exec(rawText)) !== null) {
    if (match.index > lastIndex) {
      segments.push({ type: 'text', content: rawText.slice(lastIndex, match.index) });
    }
    segments.push({
      type: 'word',
      content: match[1],
      translation: match[2],
      position: position++,
    });
    lastIndex = regex.lastIndex;
  }

  if (lastIndex < rawText.length) {
    segments.push({ type: 'text', content: rawText.slice(lastIndex) });
  }

  return segments;
}
```

7.3 答案匹配逻辑

代码块

```
// src/lib/wordMatcher.ts

export function matchChinese(input: string, answer: string): boolean {
  const normalize = (s: string) => s.trim().replace(/[, 。、 ! ? ; : ""'' () \s]/g, '');
  const i = normalize(input);
```

```
  const a = normalize(answer);
  if (i === a) return true;
  if (i.length >= 2 && (a.includes(i) || i.includes(a))) return true;
  return false;
}

export function matchEnglish(input: string, answer: string): boolean {
  return input.trim().toLowerCase() === answer.trim().toLowerCase();
}
```

7.4 JWT 工具（Web Crypto API）

代码块

```
// src/lib/jwt.ts

async function hashPassword(password: string): Promise<string> {
  const encoder = new TextEncoder();
  const hashBuffer = await crypto.subtle.digest(
    'SHA-256', encoder.encode(password + 'salt')
  );
  return Array.from(new Uint8Array(hashBuffer))
    .map(b => b.toString(16).padStart(2, '0')).join('');
}

async function signJWT(payload: object, secret: string): Promise<string> {
  const encoder = new TextEncoder();
  const header = btoa(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));
  const body = btoa(JSON.stringify(payload));
  const key = await crypto.subtle.importKey(
    'raw', encoder.encode(secret),
    { name: 'HMAC', hash: 'SHA-256' },
    false, ['sign']
  );
  const signature = await crypto.subtle.sign(
    'HMAC', key, encoder.encode(`${header}.${body}`)
  );
  const sig = btoa(String.fromCharCode(...new Uint8Array(signature)));
  return `${header}.${body}.${sig}`;
}
```

八、API 接口设计

统一响应格式

代码块

```
1   { "code": 200, "message": "success", "data": {} }
```

8.1 用户认证接口

方法	路径	说明
POST	/api/auth/register	注册（含激活码校验 + 时效计算）
POST	/api/auth/login	登录（含过期校验）
GET	/api/auth/profile	获取用户信息

8.2 业务接口

方法	路径	说明
GET	/api/stories?categoryId=1	故事列表
GET	/api/stories/[id]	故事详情
POST	/api/stories/[id]/progress	保存进度
GET	/api/bookmarks	收藏列表
POST	/api/bookmarks	添加收藏
DELETE	/api/bookmarks/[id]	取消收藏
POST	/api/bookmarks/batch-remove	批量取消
POST	/api/exam/start	开始考试
POST	/api/exam/submit	提交考试

8.3 管理后台接口

方法	路径	说明
POST	/api/admin/activation-codes/generate	批量生成激活码

GET	/api/admin/activation-codes/list	激活码列表（分页+筛选）
GET	/api/admin/activation-codes/export	导出激活码 XLS 文件
GET	/api/admin/users	用户列表

九、Cloudflare 配置

代码块

```
# wrangler.toml
name = "ielts-vocab"
compatibility_date = "2024-01-01"

[[d1_databases]]
binding = "DB"
database_name = "ielts-vocab-db"
database_id = "xxxx-xxxx-xxxx"

[[kv_namespaces]]
binding = "CACHE"
id = "xxxx-xxxx-xxxx"

[[r2_buckets]]
binding = "STORAGE"
bucket_name = "ielts-vocab-assets"
```

十、数据准备流程

整体流程

代码块

```
步骤1: 音标生成（~30分钟）
  Python 脚本 + Free Dictionary API
  词汇表.csv → 添加 phonetic_us + phonetic_uk 列

步骤2: 音频生成（~2小时）
  edge-tts + Python 脚本
  美式: en-US-AriaNeural → audio/us/{word}.mp3
  英式: en-GB-SoniaNeural → audio/uk/{word}.mp3

```

```
步骤3：上传到 R2（~30分钟）
  wrangler r2 object put 批量上传

步骤4：导入数据库（1分钟）
  wrangler d1 execute 导入词汇 + 分类 + 管理员账号
```

R2 公开访问配置

音频文件需要公开访问，通过 Cloudflare R2 绑定自定义域名实现：

- R2 Bucket 设置 → 公开访问 → 绑定子域名（如 `cdn.yourdomain.com`）
- 音频 URL：`https://cdn.yourdomain.com/audio/us/accuse.mp3`

十一、部署流程

代码块

```
# 首次部署
git clone https://github.com/yourname/ielts-vocab.git
cd ielts-vocab && pnpm install

# 创建 D1 数据库
npx wrangler d1 create ielts-vocab-db

# 执行数据库迁移
npx wrangler d1 migrations apply ielts-vocab-db --remote

# 导入初始数据
npx wrangler d1 execute ielts-vocab-db --remote --file=./seed.sql

# 部署到 Cloudflare Pages
npx wrangler pages deploy .next

# 域名：GoDaddy 买域名 → Cloudflare 添加 → 修改 NS → CNAME 指向 pages.dev
```

十二、安全方案

措施	实现方式
密码加密	SHA-256 + Salt

JWT 认证	Web Crypto API 签名
管理员权限	JWT payload 含 role，中间件校验
防暴力破解	KV 存储失败次数，5次锁定15分钟
激活码安全	同一 IP 1分钟最多3次
账号过期	登录时校验 expiresAt
HTTPS	Cloudflare 自动
SQL 注入	Drizzle ORM 参数化查询

十三、成本估算

项目	费用
GoDaddy 域名	~$10/年（约 ¥72）
Cloudflare Pages	免费
Cloudflare D1	免费（5GB）
Cloudflare R2	免费（10GB 存储，实际用 ~130MB）
Cloudflare KV	免费
Cloudflare Workers	免费
HTTPS 证书	免费
总成本	**约 ¥72/年**

文档结束